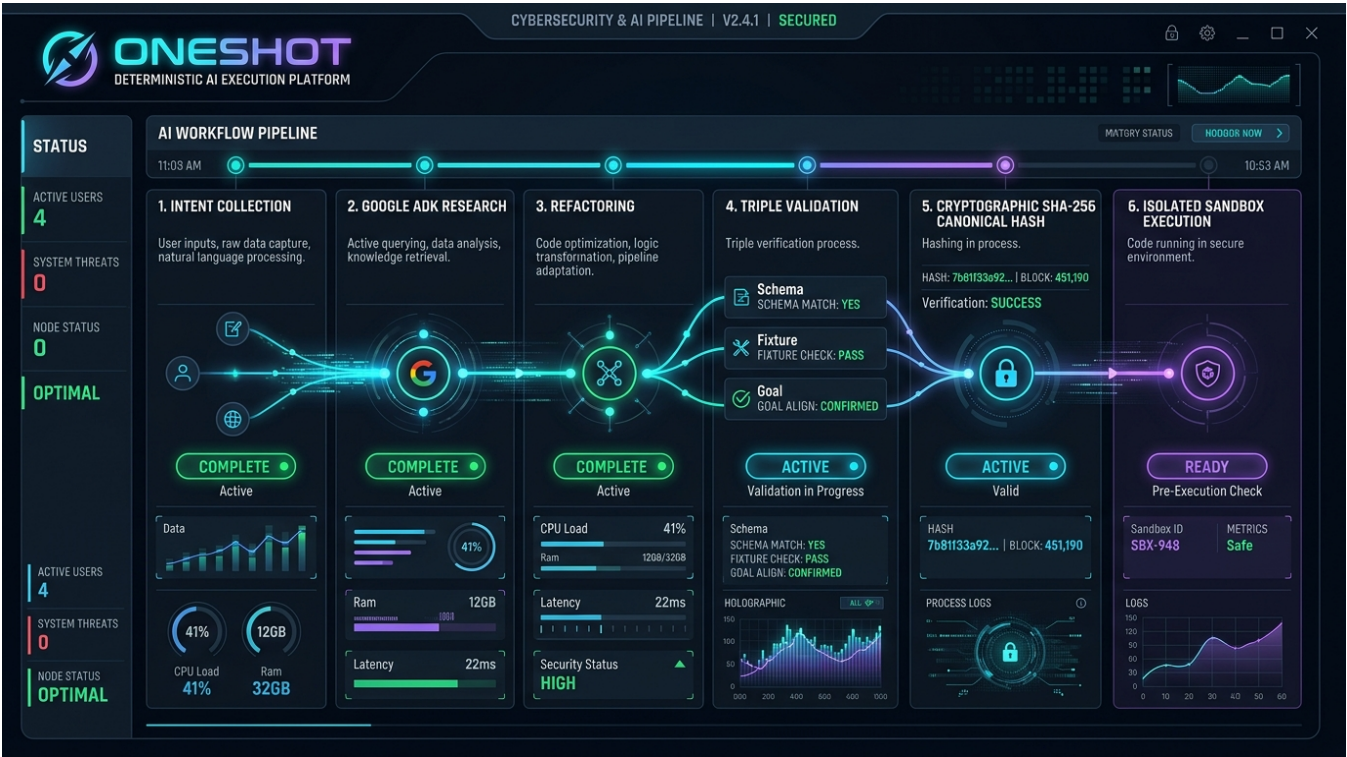


# OneShot Production E2E — Workflow Processing Specification

Comprehensive Technical Reference & Master Execution Model (v1.3.0)



## 1. Executive Summary & Core Architectural Invariants

OneShot is an enterprise-grade deterministic AI execution platform combining strict JSON Schema Draft 2020-12 validation, multi-turn conversational intent collection, verifiable AI research providers (Google ADK & Featherless), multi-stage planning, refactoring, triple validation (Schema, Fixture, Goal), RFC 8785 canonicalization, SHA-256 cryptographic hashing, isolated process/container sandbox execution, and a FastAPI Workspace API control plane.

**Fundamental Invariant:** Authority Separation is strictly enforced:  $ROLE \neq SKILL \neq TOOL \neq WORKFLOW$ . Every phase produces schema-validated artifacts with monotonic sequence ordering and W3C trace context.

## 2. Canonical 27-Phase Execution Trace

The end-to-end execution path progresses through 27 deterministic phases from user chat to isolated sandbox execution:

| Phase | Phase Name           | Handler / Component                 | Output Artifact / Invariant                  |
|-------|----------------------|-------------------------------------|----------------------------------------------|
| 1-3   | Intent & Prompt Gate | backend/intent/intent-collection.ts | Intent(id) revision → Prompt(id) work order. |

|       |                               |                                       |                                                           |
|-------|-------------------------------|---------------------------------------|-----------------------------------------------------------|
| 4-7   | Researcher & Draft Validation | backend/role/researcher/workflow.ts   | Structured Research Draft → 6 canonical bundle artifacts. |
| 8-9   | Planner Audit                 | backend/role/planner/workflow.ts      | 7-area inspection → Audit(audit_id) findings.             |
| 10-12 | Refactor Engine               | backend/role/refactor/workflow.ts     | Preserves identical plan_id → Revision increment.         |
| 13-14 | Gap Analysis & Recheck        | backend/role/gap-analysis/workflow.ts | Branch verification → proves gap_0: true (PASSED).        |
| 15-16 | Evaluation Matrix             | backend/role/evaluation/workflow.ts   | 9-point criteria check → Evaluation artifact.             |
| 17-20 | Triple Validation Engine      | backend/workflow/triple-validation.ts | Schema + Fixture + Goal validators → all_valid: true.     |
| 21-22 | Confirmed Packaging           | backend/workflow/confirmation.ts      | ConfirmedCore (10 artifacts) → ConfirmedPackage.          |
| 23-25 | RFC 8785 & SHA-256 Hash       | backend/workflow/hash.ts              | Python JCS canonicalization → SHA-256 Hash equality.      |
| 26    | Canonical State DONE          | backend/runtime/workflow-runtime.ts   | HashProof snapshot → DONE terminal state.                 |
| 27    | Sandbox Execution Boundary    | backend/sandbox/sandbox-service.ts    | Cryptographic admission → hash_sandbox == HASH.           |

### 3. Source-of-Truth Workflow Tree Specification

Below is the hierarchical workflow execution tree specified in docs/WORKFLOW\_TREE:

```

START → USER → CHAT (Intent, Outcome, Requirements, Constraints)
↓
INTENT READY → GENERATOR → Prompt_id [Record: JOB_ID ✓]
↓
RESEARCHER (agent/researcher/workflow.ts)
  ■■■ LOAD SKILL: agent/researcher/skill/SKILL.md
  ■■■ Multi-Provider AI (Google ADK Gemma 2 / Featherless Gemma 4)
  ■■■ OUTPUT: Researcher(id) [plan_id, schema_id, fixture_id, goal_id, validation_id]
↓
PLANNER (agent/planner/workflow.ts)
  ■■■ 7-Area Audit: sufficiency, file coverage, structure, design, goals, fixtures, conflicts
  ■■■ OUTPUT: Audit(audit_id)
↓
REFACTOR / REFINEMENT (agent/refactor/workflow.ts)
  ■■■ Refine SAME plan_id (preserves identity & provenance)
  ■■■ OUTPUT: Updated plan_id (Revision 2)
↓
GAP ANALYSIS (agent/gap-analysis/workflow.ts)
  ■■■ Branch verification & gap correction
  ■■■ OUTPUT: FINAL plan_id with gap_0: true (PASSED)
↓
EVALUATION (agent/evaluation/workflow.ts)
  ■■■ 9-Point evaluation matrix verification
↓
TRIPLE VALIDATION (validation/triple_validation.py)
  ■■■ Schema Validation (Draft 2020-12)
  ■■■ Fixture Validation (Assertion operators)
  ■■■ Goal Validation (Success criteria alignment)
  ■■■ OUTPUT: Final_Confirmed_Validation(hash) → all_valid: true
↓
CONFIRMATION & CANONICAL HASH (RFC 8785 JCS + SHA-256)
  ■■■ ConfirmedPackage → created_hash == recomputed_hash → HASH ✓
↓
BUILDER / SANDBOX EXECUTION (backend/sandbox/sandbox-service.ts)
  ■■■ Verified Admission → Isolated Execution → hash_sandbox == HASH → DONE ✓

```

### 4. Triple Validation & Cryptographic Proof Verification

To guarantee total determinism and execution safety, OneShot implements three decoupled validation engines in Python:

- **1. Schema Validation:** Proves that the plan, researcher output, fixtures, and contracts strictly conform to Draft 2020-12 JSON Schemas without extraneous or missing fields.
- **2. Fixture Validation:** Evaluates all assertion operators (exists, equals, matchesSchema, contains, greaterThan) against deterministic test inputs.
- **3. Goal Validation:** Traces all plan steps against declared success criteria and requirements to prove complete goal achievement.
- **4. RFC 8785 Canonicalization & SHA-256:** JCS serializes the exact core bytes in deterministic UTF-8 order, generating an immutable 64-character SHA-256 cryptographic hash.

### 5. Hardened Sandbox Isolation & Admission Gate

The external sandbox runtime provides a zero-trust execution boundary with three strict guarantees:

- **Cryptographic Admission:** Rejects any tampered, modified, or malformed plan package before execution starts.
- **Isolated Execution:** Runs code under non-root users with strict CPU limits, memory quotas, total byte-write limits, and isolated DENY\_ALL network policies.
- **Post-Execution Hash Parity:** Asserts that the executed sandbox package matches the immutable pre-execution hash: `hash_sandbox == HASH`.

## 6. Software License & Governance

The OneShot Production E2E platform and all associated workflow processing engines, contracts, skills, and tools are licensed under the **Apache License, Version 2.0**. See LICENSE and NOTICE for full terms.